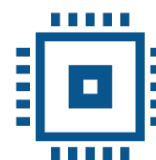




Universitatea *Transilvania* din Brașov
Facultatea de Inginerie Electrică și Știința
Calculatoarelor
Departamentul Automatică și Tehnologia Informației



Temă de semestru P00/PCLP3

Jipa Stefan Ciprian
Gr. 4LF442, AIA II

BRAȘOV, 2026

Cuprins

<u>1. Titlul proiectului.....</u>	<u>2</u>
<u>1.1. Enunț.....</u>	<u>2</u>
<u>1.2. Rezolvare proiect.....</u>	<u>2</u>
<u>1.3. Rezultate.....</u>	<u>3</u>

1. Titlul proiectului

Laborator 9

1.1. Enunț

Exercițiul 1 – Extinderea jurnalului Se cere extinderea aplicației „Gestionar cont bancar” cu următoarele funcționalități: ▪ adăugarea unui buton „Salvează tranzacțiile” care scrie întregul istoric de tranzacții într- un fișier text (tranzactii.csv), câte o linie pe tranzacție, folosind clasa File. Se va gestiona corect blocul try- catch-finally pentru a trata UnauthorizedAccessException și IOException ; ▪ adăugarea unui buton „Încarcă tranzacțiile” care citește fișierul precedent și repopulează ListBox - ul cu istoricul. 17.2.2. Exercițiul 2 – Excepție personalizată nouă Se cere: ▪ definirea unei excepții LimitaZilnicaDepasitaException (derivată din InvalidOperationException) care conține proprietățile LimitaZilnica și SumaTotalaRetrasaAstazi; ▪ modificarea clasei ContBancar pentru a limita totalul retragerilor dintr - o zi la 1000 RON și a arunca noua excepție atunci când limita este depășită; ▪ adăugarea unui bloc catch corespunzător în Form1.cs care afișează un mesaj dedicat, inclusiv valorile din proprietățile excepției; ▪ extinderea testelor manuale cu scenarii care depășesc limita și verifică mesajul afișat. 17.2.3. Exercițiul 3 – Depanare în Visual Studio Fără a modifica codul aplicației, se cere efectuarea următoarelor activități de depanare și documentarea rezultatelor într - un document text scurt : ▪ plasarea unui breakpoint condiționat pe linia de adunare a sumei în metoda Depune , care să oprească execuția doar atunci când _sold > 10000; ▪ utilizarea ferestrelor Locals și Watch pentru a inspecta valorile câmpului _sold și a numărului de tranzacții pe parcursul mai multor depuneri consecutive; ▪ urmărirea apelului Retrage cu Step Into (F11) și identificarea punctului exact în care se aruncă FondInsuficientException; ▪ activarea opririi la excepția FondInsuficientException din Exception Settings și observarea momentului în care debugger - ul oprește execuția.

1.2. Rezolvare proiect

lab9.java

```
using System; using System.Collections.Generic; using System.Drawing; using System.Globalization;
using System.IO; using System.Linq; using System.Windows.Forms;
namespace laborator9 { // ===== // MODELE ȘI EXCEPȚII
(Exercițiul 2 și Logica de Bază) // =====
public class Tranzactie
{
    public DateTime Data { get; set; }
    public string Tip { get; set; }
    public decimal Suma { get; set; }
    public string Comentariu { get; set; }
```

```
public override string ToString()
{
    return $"{Data:g} | {Tip} | {Suma:F2} RON | {Comentariu}";
}

// Metode utile pentru Exercițiul 1 (Serializare/Deserializare text
simplă)
public string ToCsv()
{
    return $"{Data:0},{Tip},
{Suma.ToString(CultureInfo.InvariantCulture)},{Comentariu}";
}

public static Tranzactie FromCsv(string csvLine)
{
    var parts = csvLine.Split(',');
    return new Tranzactie
    {
        Data = DateTime.Parse(parts[0], null,
DateTimeStyles.RoundtripKind),
        Tip = parts[1],
        Suma = decimal.Parse(parts[2], CultureInfo.InvariantCulture),
        Comentariu = parts.Length > 3 ? parts[3] : ""
    };
}

public class SumaInvalidaException : Exception
{
    public decimal SumaPrimita { get; }
    public SumaInvalidaException(string message, decimal suma) :
base(message)
    {
        SumaPrimita = suma;
    }
}

public class FondInsuficientException : Exception
{
    public decimal SoldCurent { get; }
    public FondInsuficientException(string message, decimal sold) :
base(message)
    {
        SoldCurent = sold;
    }
}

// Excepție creată pentru Exercițiul 2
public class LimitaZilnicaDepasitaException : InvalidOperationException
{
    public decimal LimitaZilnica { get; }
    public decimal SumaTotalaRetrasaAstazi { get; }
```

```

        public LimitaZilnicaDepasitaException(decimal limita, decimal retrasa,
string message)
            : base(message)
        {
            LimitaZilnica = limita;
            SumaTotalaRetrasaAstazi = retrasa;
        }
    }

public class ContBancar
{
    public decimal Sold { get; private set; }
    public List<Tranzactie> Istoric { get; private set; } = new
List<Tranzactie>();

    public void Depune(decimal suma, string comentariu = "")
    {
        if (suma <= 0)
            throw new SumaInvalidaException("Suma de depus trebuie să fie
pozitivă.", suma);

        Sold += suma;
        Istoric.Add(new Tranzactie { Data = DateTime.Now, Tip = "Depunere",
Suma = suma, Comentariu = comentariu });
    }

    public Tranzactie Retrage(decimal suma, string comentariu = "")
    {
        if (suma <= 0)
            throw new SumaInvalidaException("Suma de retras trebuie să fie
pozitivă.", suma);
        if (suma > Sold)
            throw new FondInsuficientException("Fonduri insuficiente.",
Sold);

        // Folosim 'tr' aici pentru a nu intra în conflict
        decimal retrageriAzi = Istoric
            .Where(tr => tr.Tip == "Retragere" && tr.Data.Date ==
DateTime.Today)
            .Sum(tr => tr.Suma);

        if (retrageriAzi + suma > 1000m)
        {
            throw new LimitaZilnicaDepasitaException(1000m, retrageriAzi,
$"Limita zilnică de 1000 RON a fost depășită! Ați retras
deja {retrageriAzi} RON astăzi.");
        }

        Sold -= suma;

        // Am redenumit variabila din 't' în 'tranzactieNoua'
    }
}

```

```
        var tranzactieNoua = new Tranzactie { Data = DateTime.Now, Tip =
"Retragere", Suma = suma, Comentariu = comentariu };
        Istoric.Add(tranzactieNoua);

        return tranzactieNoua;
    }

    public void Reseteaza()
    {
        Sold = 0;
        Istoric.Clear();
    }

    // Metodă ajutătoare pentru Exercițiul 1
    public void IncarcaIstoricDinFisier(List<Tranzactie> istoricIncarcat)
    {
        Istoric = istoricIncarcat;
        Sold = Istoric.Where(t => t.Tip == "Depunere").Sum(t => t.Suma) -
            Istoric.Where(t => t.Tip == "Retragere").Sum(t => t.Suma);
    }
}

// =====
// INTERFAȚA GRAFICĂ (Fără designer)
// =====

public class Form1 : Form
{
    private ContBancar _cont = new ContBancar();
    private Label lbSold;
    private TextBox tbSuma;
    private TextBox tbComentariu;
    private ListBox lbxTranzactii;
    private ListBox lbxErori;

    public Form1()
    {
        Text = "Gestionar Cont Bancar - Lab 9";
        Size = new Size(620, 520);
        StartPosition = FormStartPosition.CenterScreen;

        // Inițializare controale
        lbSold = new Label { Text = "Sold: 0,00 RON", Font = new Font(Font,
FontStyle.Bold), Location = new Point(20, 20), AutoSize = true };

        Label lblSuma = new Label { Text = "Sumă (RON):", Location = new
Point(20, 60), AutoSize = true };
        tbSuma = new TextBox { Location = new Point(120, 57), Width =
100 };

        Label lblComentariu = new Label { Text = "Comentariu:", Location =
new Point(20, 90), AutoSize = true };
```

```

        tbComentariu = new TextBox { Location = new Point(120, 87), Width =
        200 };

        Button btnDepune = new Button { Text = "Depune", Location = new
        Point(20, 130), Width = 90 };
        btnDepune.Click += BtnDepune_Click;

        Button btnRetrage = new Button { Text = "Retrage", Location = new
        Point(120, 130), Width = 90 };
        btnRetrage.Click += BtnRetrage_Click;

        Button btnReset = new Button { Text = "Resetează contul", Location
        = new Point(220, 130), Width = 120 };
        btnReset.Click += BtnReset_Click;

        // Butoane adăugate pentru Exercițiul 1
        Button btnSalveaza = new Button { Text = "Salvează tranzacțiile",
        Location = new Point(20, 170), Width = 150 };
        btnSalveaza.Click += BtnSalveaza_Click;

        Button btnIncarca = new Button { Text = "Încarcă tranzacțiile",
        Location = new Point(190, 170), Width = 150 };
        btnIncarca.Click += BtnIncarca_Click;

        Label lblIstoric = new Label { Text = "Istoric tranzacții",
        Location = new Point(20, 210), AutoSize = true, Font = new Font(Font,
        FontStyle.Bold) };
        lbxTranzactii = new ListBox { Location = new Point(20, 230), Width
        = 560, Height = 100 };

        Label lblErori = new Label { Text = "Erori", Location = new
        Point(20, 340), AutoSize = true, Font = new Font(Font, FontStyle.Bold) };
        lbxErori = new ListBox { Location = new Point(20, 360), Width =
        560, Height = 100, ForeColor = Color.Red };

        // Adăugare controale pe formular
        Controls.AddRange(new Control[] {
            lbSold, lblSuma, tbSuma, lblComentariu, tbComentariu,
            btnDepune, btnRetrage, btnReset, btnSalveaza, btnIncarca,
            lblIstoric, lbxTranzactii, lblErori, lbxErori
        });
    }

    private decimal CitesteSuma()
    {
        if (string.IsNullOrEmpty(tbSuma.Text))
            throw new SumaInvalidaException("Completați câmpul Sumă.", 0);

        string text = tbSuma.Text.Trim().Replace(',', '.');
        if (!decimal.TryParse(text, NumberStyles.Number,
        CultureInfo.InvariantCulture, out decimal val))
            throw new FormatException();
    }

```

```
        return val;
    }

    private void AfiseazaEroare(string mesajUtilizator, Exception ex)
    {
        lbxErori.Items.Add($"{DateTime.Now:HH:mm:ss} - {mesajUtilizator}");

        try
        {
            File.AppendAllText("erori.log", $"{DateTime.Now}: {ex.Message}\n");
        }
        catch { /* Ignorăm erorile de logare internă pentru a nu opri
programul */ }
    }

    private void ActualizeazaInterfata()
    {
        lbSold.Text = $"Sold: {_cont.Sold:F2} RON";
        lbxTranzactii.Items.Clear();
        foreach (var t in _cont.Istoric)
            lbxTranzactii.Items.Add(t);
    }

    private void CurataCampuri()
    {
        tbSuma.Clear();
        tbComentariu.Clear();
        tbSuma.Focus();
    }

    private void BtnDepune_Click(object sender, EventArgs e)
    {
        try
        {
            decimal suma = CitesteSuma();
            _cont.Depune(suma, tbComentariu.Text);
            ActualizeazaInterfata();
            CurataCampuri();
        }
        catch (SumaInvalidaException ex) { AfiseazaEroare(ex.Message,
ex); }
        catch (FormatException ex) { AfiseazaEroare("Suma introdusă nu este
un număr valid.", ex); }
        catch (Exception ex) { AfiseazaEroare("Eroare neașteptată: " +
ex.Message, ex); }
    }

    private void BtnRetrage_Click(object sender, EventArgs e)
    {
        try
```

```
{
    decimal suma = CitesteSuma();
    _cont.Retrage(suma, tbComentariu.Text);
    ActualizeazaInterfata();
    CurataCampuri();
}
// Tratarea excepției personalizate de la Exercițiul 2
catch (LimitaZilnicaDepasitaException ex)
{
    AfiseazaEroare($"Limită depășită! Ați retras deja
{ex.SumaTotalaRetrasaAstazi} azi, dintr-o limită de {ex.LimitaZilnica}.",
ex);
}
catch (FondInsuficientException ex) { AfiseazaEroare($"Fond
insuficient (sold: {ex.SoldCurent:F2}).", ex); }
catch (SumaInvalidaException ex) { AfiseazaEroare(ex.Message,
ex); }
catch (FormatException ex) { AfiseazaEroare("Suma introdusă nu este
un număr valid.", ex); }
catch (Exception ex) { AfiseazaEroare("Eroare neașteptată: " +
ex.Message, ex); }
}

private void BtnReset_Click(object sender, EventArgs e)
{
    _cont.Reseteaza();
    lbxErori.Items.Clear();
    ActualizeazaInterfata();
}

// =====
// EXERCIȚIUL 1: Extinderea jurnalului
// =====

private void BtnSalveaza_Click(object sender, EventArgs e)
{
    try
    {
        var lines = _cont.Istoric.Select(t => t.ToCsv());
        File.WriteAllLines("tranzactii.csv", lines);
        MessageBox.Show("Tranzacțiile au fost salvate cu succes în
tranzactii.csv!", "Salvare completă", MessageBoxButtons.OK,
MessageBoxIcon.Information);
    }
    catch (UnauthorizedAccessException ex) { AfiseazaEroare("Nu aveți
permisiuni pentru a scrie fișierul.", ex); }
    catch (IOException ex) { AfiseazaEroare("Eroare de I/O la scrierea
fișierului.", ex); }
    catch (Exception ex) { AfiseazaEroare("Eroare neașteptată la
salvare: " + ex.Message, ex); }
}
```

```
private void BtnIncarca_Click(object sender, EventArgs e)
{
    try
    {
        if (!File.Exists("tranzactii.csv"))
        {
            MessageBox.Show("Fișierul tranzactii.csv nu există. Salvați  
mai întâi.", "Eroare", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        var lines = File.ReadAllLines("tranzactii.csv");
        List<Tranzactie> istoricNou = new List<Tranzactie>();

        foreach (var line in lines)
        {
            if (!string.IsNullOrEmpty(line))
            {
                istoricNou.Add(Tranzactie.FromCsv(line));
            }
        }

        _cont.IncarcaIstoricDinFisier(istoricNou);
        ActualizeazaInterfata();
        MessageBox.Show("Tranzacțiile au fost încărcate cu succes!",
            "Încărcare completă", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    catch (UnauthorizedAccessException ex) { AfiseazaEroare("Nu aveți  
permisiuni pentru a citi fișierul.", ex); }
    catch (IOException ex) { AfiseazaEroare("Eroare de I/O la citirea  
fișierului.", ex); }
    catch (Exception ex) { AfiseazaEroare("Eroare neașteptată la  
încărcare: " + ex.Message, ex); }
}

// =====
// PUNCTUL DE INTRARE ÎN APLICAȚIE
// =====
static class Program
{
    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new Form1());
    }
}
```

1.3. Rezultate

Gestionar Cont Bancar - Lab 9

Sold: 3000.00 RON

Sumă (RON):

Comentariu:

Istoric tranzacții

5/24/2026 7:19 PM | Depunere | 3500.00 RON | Suma
5/24/2026 7:19 PM | Retragere | 500.00 RON | retragere

Erori

19:19:22 - Completați câmpul Sumă.